

Analysis: Falling Diamonds

This was a tricky problem, and solving the small often made the difference between advancing and not - in particular, solving all of this problem was enough to advance; as was solving the small of this problem and the large of Osmos.

The small case

For the small case, we had only twenty diamonds to deal with in each test case. Note that every diamond will do something non-deterministic at most once, when it falls point down on the point of another diamond (when it can slide in one of two directions). This means that we will have at most 2^{20} different things that can happen when the diamonds fall, so we can simply try to enumerate all of them (note that there is actually less possible paths - for instance the first, fifth and sixth will never have a choice).

This is actually a bit tricky to do, since in each branch we have to keep track of what is the probability of reaching this branch — it is not only dependent on how many diamonds we processed so far, but rather on how many diamonds had a choice so far. After this is done, we need to figure out in how many of all the options the place we're interested in did get a diamond and add them all up. All this is not easy to get right, but it is doable, as over 900 of our contestants proved!

The large case

For the large test case, we will need to be smarter — simulating all the options is obviously not a choice for 10^6 diamonds. We will begin with the following observation:

The diamonds fall in layers. First a diamond falls at $(0, 0)$, then diamonds fall into positions with $|X| + |Y| = 2$, only after all five of these are filled the positions with $|X| + |Y| = 4$ start filling, only after them the $|X| + |Y| = 6$ start filling, and so on.

Indeed, note that the diamond sliding to one side does not change the layer it is in, since it always starts sliding in some $(0, 2k)$ position, and the $(0, 2k)$ position is always the last in a layer to be filled.

Thus, when N diamonds fall, the only uncertainty as to how they shape up is in the last layer, and this is what we have to calculate. If the place we are considering is not in this layer, we can respond immediately. Thus, we have only to figure out probabilities in the last layer.

The dynamic programming approach

First let's estimate how large the last layer can be. If we have at most a million diamonds, one can calculate there will be no more than 710 layers. When diamonds fall, the state of the layer can be described by two numbers — how many diamonds are on the left of the center (with negative X), and how many are to the right (we assume here there aren't enough diamonds to fill this layer, so the top spot with $X = 0$ will stay empty). This means that when the diamonds drop, there are roughly 500,000 different states to consider.

One can approach to this problem is dynamic programming. For each of the states possible for the last layer, we calculate the probability of reaching this state when the appropriate number of

diamonds has dropped (each state determines the number of diamonds uniquely).

A formulaic approach

One can also notice that what matters is how many diamonds of the ones that hit the top decide to go left, and how many to go right. Which diamonds exactly are those does not matter for the final state. Thus, we can precalculate binomial coefficients (or rather binomial coefficients divided by 2^D , where D is the number of diamonds falling into the layer), and — once we know which layer we're looking — sum up the options that lead to a diamond falling into the right place.

Analysis: Garbled Email

There are multiple approaches possible to this problem. For the small test case, one can use a [dynamic programming](#) approach to calculate for each prefix of the given word what's the smallest number of substitutions needed to form this word so that the last substitution was k characters ago, for $k = 1, 2, 3, \dots$

For example, for the word "codejam", we will find that "c" cannot be formed without a substitution, but can be formed (for instance from "a") by a substitution 1 character away. We find this by going over all dictionary words. Then, we go over all dictionary words to try and form "co" (we can do this, for instance, from "do" with one substitution 2 characters ago). We can also consider one letter words to extend the "c" we already know how to form, but this won't work, since "o" isn't a word, and we're too near to the last substitution. Next goes "cod", which actually is a word, so can be formed with zero substitutions. Next goes "code" — for this we have a number of choice, like combining the "c" we know how to form and "ode", or the "cod" and a one substitution to form "e" from "a", or — the best one, since requiring no substitutions — just using the word "code".

In this fashion for each prefix and each distance of the last substitution we can find out what's the least number of substitutions needed to form this prefix by looking at all smaller prefixes (including the empty one), all smaller dictionary words, and figuring out whether we can combine them.

For the large test case, we can't afford to go over the whole dictionary that often. So, we start by building a hash table. For each dictionary word, we insert that word into the hash table, and also insert the word with each possible set of changed letters in the word replaced by '*' characters.

For example, for the word "coders", we store in the hash table:

- coders
- *oders
- c*ders
- co*ers
- cod*rs
- code*s
- coder*
- *oder*

Next we use dynamic programming to build a table that contains, for each prefix of the email and location within that prefix of the last changed letter, the minimum number of changes required to transform a sequence of dictionary words into the prefix, if it is possible. (To save time, we can merge together all the states for a prefix where the last changed letter was 5 or more positions from the end of the prefix, because if the last changed letter is more than 4 positions back, it doesn't matter how much more.)

Each of the states where this is possible corresponds to a partial solution. We consider each possible way of adding one more word to create a longer partial solution. To do this, we try each combination of:

- The length of the next word, L ($1 \leq L \leq 10$).
- Each possible set of positions of changed letters in the next word, S .

For each of these combinations, we construct a string by taking the next **L** letters of the original email, and changing the positions in **S** to '*' characters. Then we search for this string in the hash table. If we find it, we can update the table to reflect a partial solution formed by appending that word.

The answer is the minimum number of changes found to produce the prefix that is the entire email.

Analysis: Osmos

We can simplify the problem by making the following observation:

There is an optimal solution where Armin chooses to absorb motes in order from smallest to largest (skipping removed motes).

If Armin's solution absorbs mote X before mote Y, and mote X is larger than mote Y, then he could change his solution to absorb Y right before X, without needing to perform any extra "add" or "remove" operations. So if Armin has an optimal solution, we can always change it into an optimal solution that absorbs motes in order of size.

Now we can limit our search to solutions that absorb motes in order of size. We could use a dynamic programming algorithm where the state is the number of motes considered, and Armin's mote's current size or the number of operations performed, but there is a simpler algorithm based on the following observation:

In an optimal solution, if Armin removes a mote, he also removes all motes of equal or greater size.

To see this, consider a solution where there exist motes X and Y where X is smaller than or equal in size to Y, X is removed, and Y is absorbed. X could instead be absorbed immediately before Y is absorbed, which would save an operation by not removing X. So the solution cannot be optimal.

So to find an optimal solution, we only need to consider $N+1$ cases -- those where we try to absorb 0, 1, ... N of the original motes and remove the remainder.

To find how many operations are needed for each of these cases, we simulate Armin trying to absorb each mote in turn. If Armin's mote is not yet large enough to absorb the next mote, we add motes of size one less than Armin's mote's current size and absorb them, until Armin's mote is large enough.

This solution takes $O(N^2)$ time to run as written, which is fast enough given the input size limits. There is a small adjustment to it that will make it linear, though. Can you see it?

One final case to handle is when Armin's mote is of size 1, and so is unable to absorb any motes at all. We were generous and added this as a case in the sample input!

This mechanics for this problem were inspired by [Osmos](#) by [Hemisphere Games](#).